

РиэлтиАпп — система управления арендой недвижимости

Веб-приложение для автоматизации учёта объектов, договоров, платежей и заявок — цель: создать удобный MVP для частных владельцев и управляющих.



Made with GAMMA



Актуальность и цель

Актуальность

Ручное управление ареной растёт в сложности при увеличении числа объектов; автоматизация снижает ошибки и экономит время.

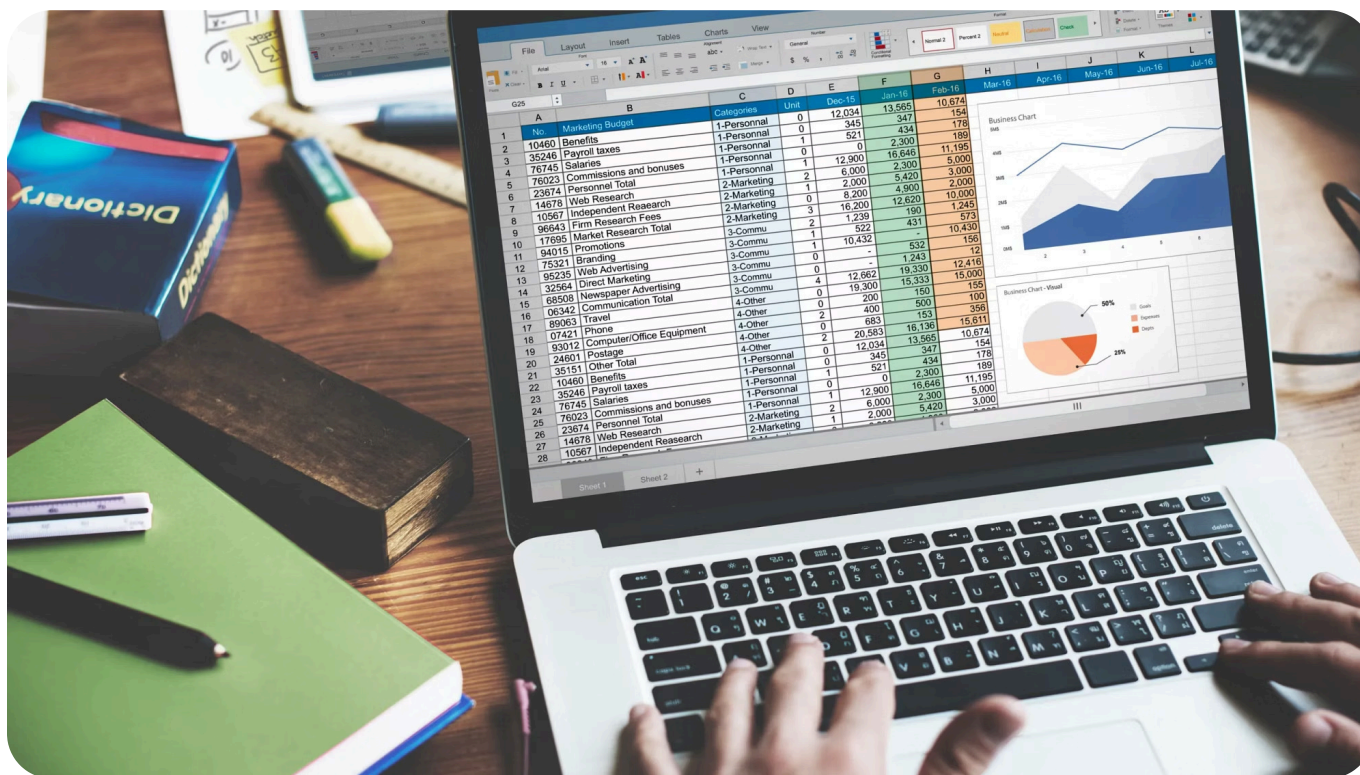
Цель

Разработать информационную систему «РиэлтиАпп» для управления арендой и формирования пассивного дохода владельцев.

Задачи

Анализ предметной области, требования, выбор стека, проект, реализация и тестирование.

Анализ предметной области — ключевые сущности



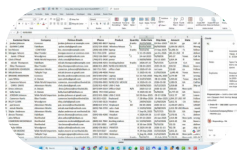
Участники и объекты

- Владельцы, арендаторы, управляющие
- Сущности: объекты, договора, платежи, заявки

Ключевые процессы

- Добавление объектов, поиск арендаторов, заключение договоров
- Контроль сроков, учёт платежей, обработка заявок

Проблемы существующих подходов



Таблицы и заметки

Недостаток доступа, высокая вероятность ошибок, отсутствие автоматизации.



Крупные облачные сервисы

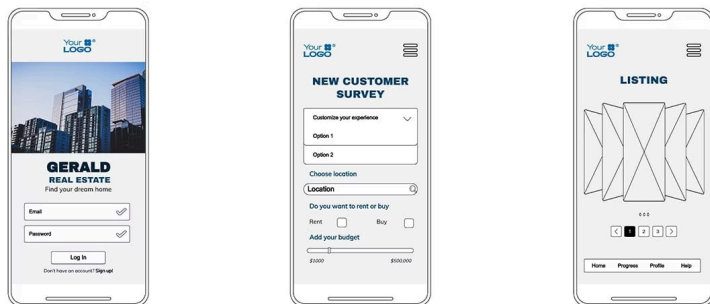
Избыточный функционал, стоимость и сложность для частных владельцев.

Необходим упрощённый, но функциональный специализированный продукт.

Требования к системе (MVP)

Real Estate App

Wireframe Whiteboard



01

Роли

Единая модель пользователя:
владелец и арендатор по
контексту.

02

Функции

Регистрация/авторизация,
управление объектами, договоры,
платежи, заявки, статистика.

03

Платежи и заявки

История платежей, статусная модель (создан, подтверждён, отклонён),
создание и обработка заявок.

Технологический стек — выбор



Сервер

FastAPI — выбран за производительность, асинхронность и автодоку.



Клиент

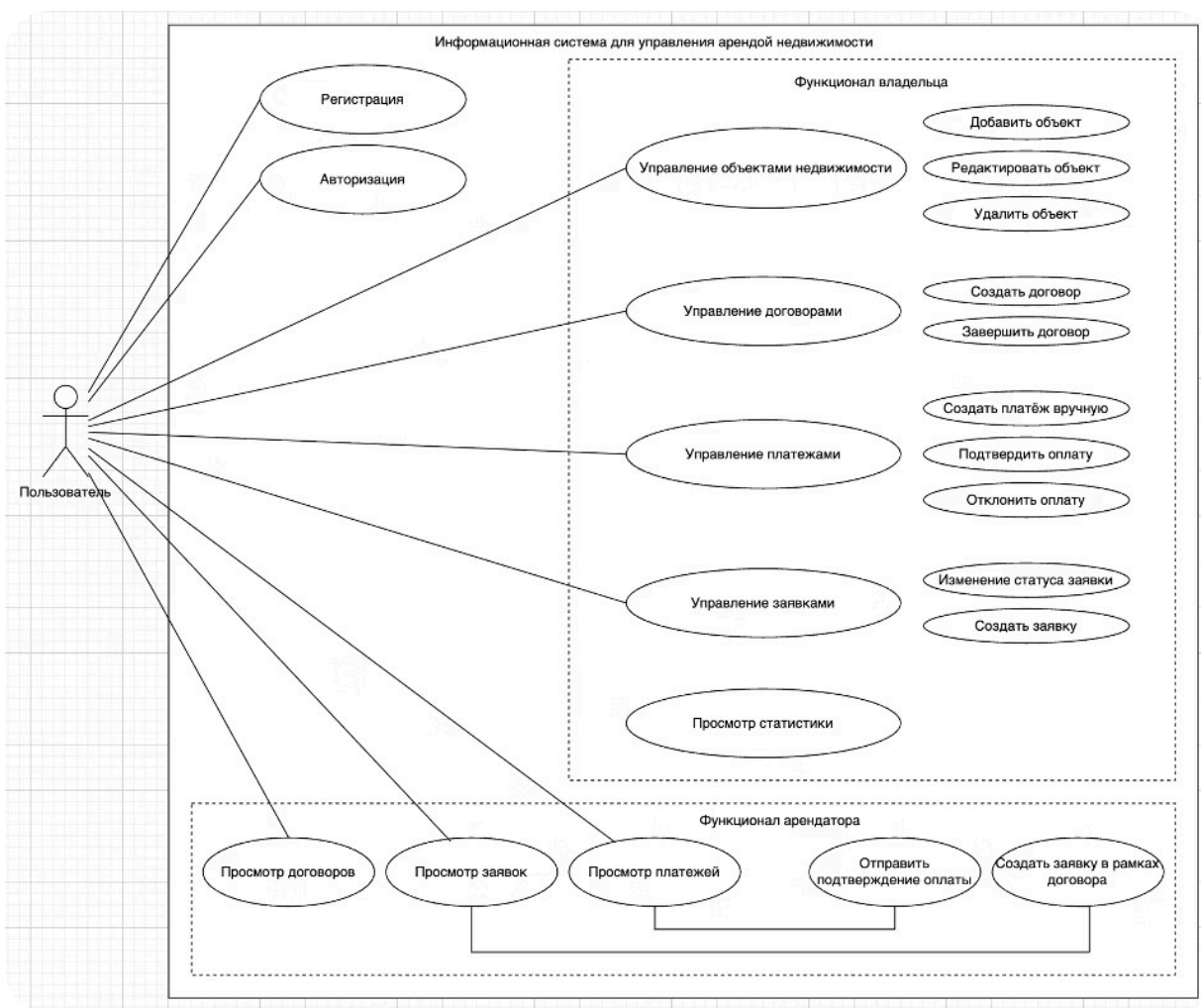
Vue 3 + TypeScript (Pinia, Vue Router) — быстрый SPA, удобство разработки.



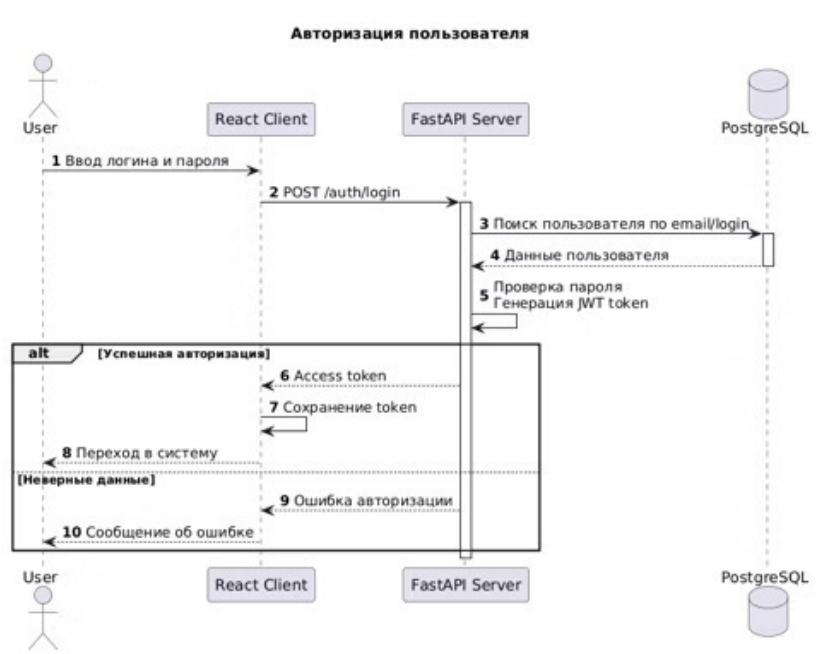
База данных

PostgreSQL — надёжность, масштабируемость, поддержка связей между сущностями.

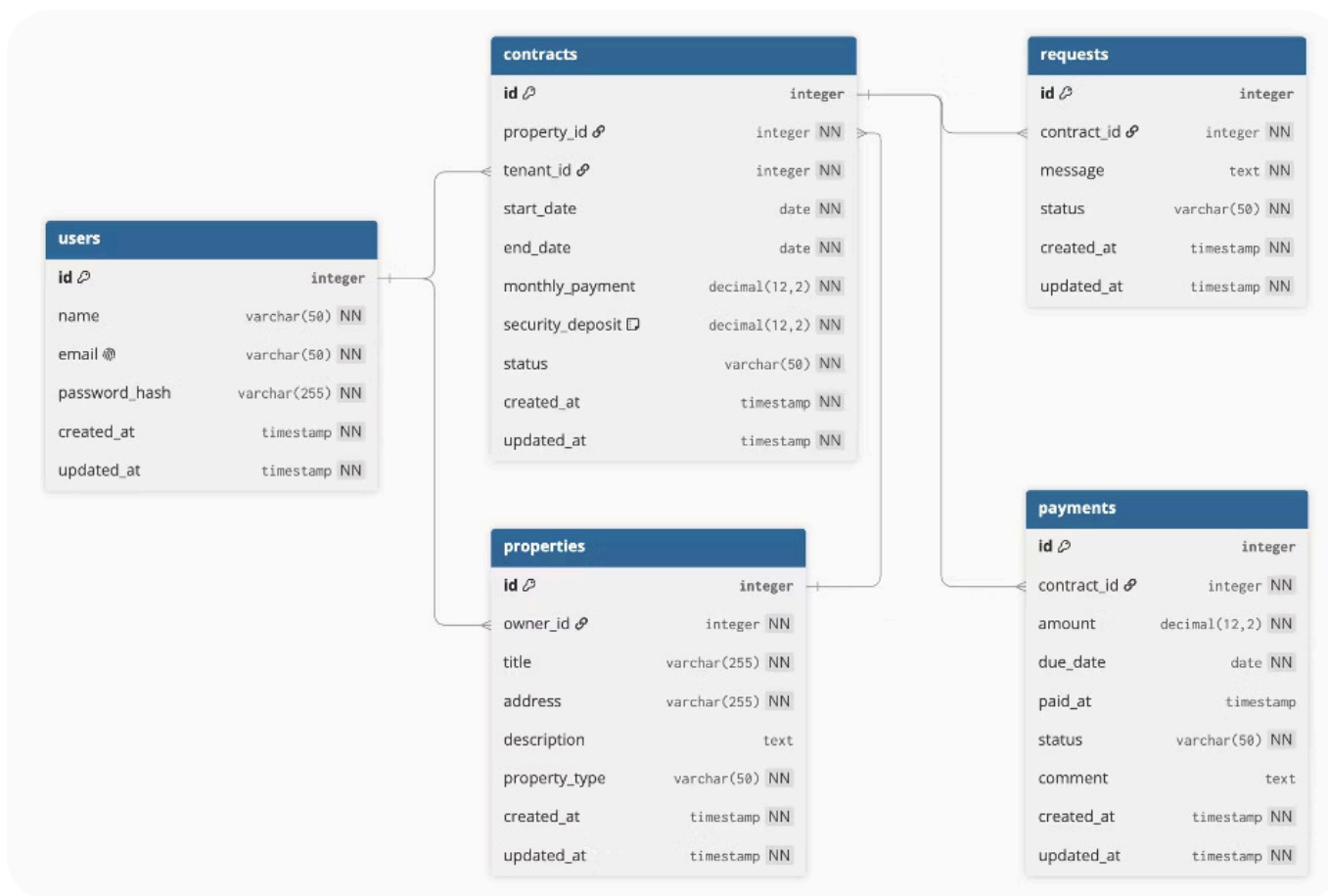
Архитектура и диаграммы



Диаграммы отражают: пользователь как актер; функционал владельца (объекты, договоры, платежи, заявки, статистика) и арендатора (просмотр договоров, платежей, создание заявок, подтверждение оплат).



Модель данных (ERD)



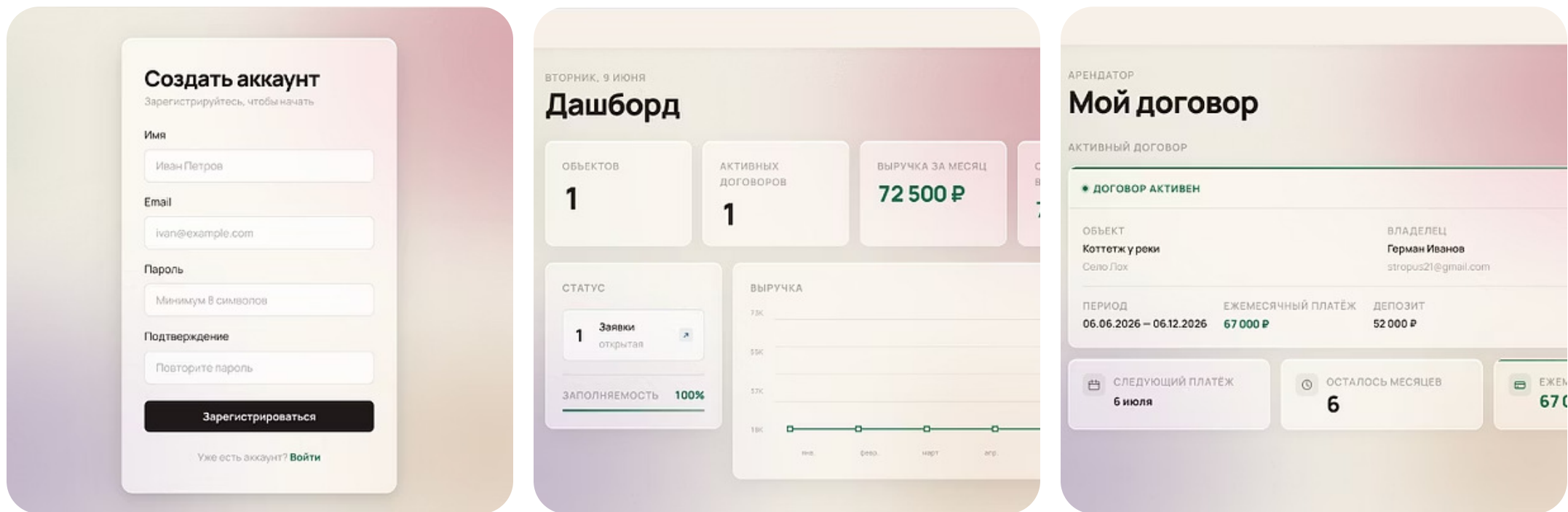
Основные сущности

Users, Properties,
Contracts, Payments,
Requests – связи 1:N через
Contract.

Ключевые связи

Property → Contract →
(Payments, Requests); User
определяется контекстом
(владелец/арендатор).

Интерфейс и статистика

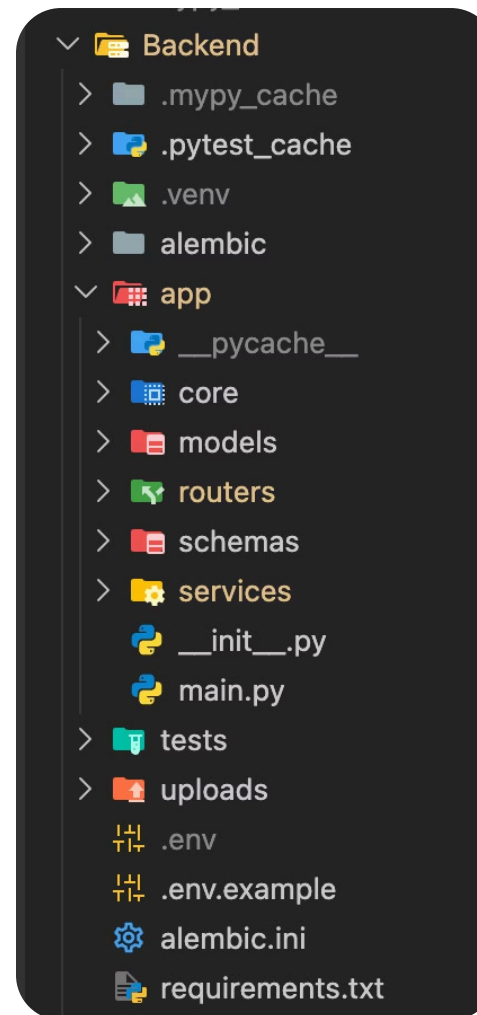


Ключевые отображения: главная владельца (карточки объектов, KPI), платежи (статусы), заявки (карточки и смена статуса), статистика (доходы, просрочки).

Разработка, тестирование и ввод в эксплуатацию

Реализация: Backend (FastAPI, SQLAlchemy, Alembic), Frontend (Vue 3 + TS, Pinia), PostgreSQL. Установка через .env, миграции, запуск uvicorn и фронтенда.

Тестирование: сценарии регистрации, авторизации, создания объектов/договоров, подтверждения платежей и обработки заявок — все ключевые тесты пройдены.



Для запуска: создать БД, настроить DATABASE_URL, выполнить alembic upgrade head, запустить uvicorn и npm start для фронтенда.

Заключение

РиэлтиАпп завершён как готовое к использованию MVP, которое объединяет управление объектами недвижимости, договорами и заявками в одном удобном решении.

Основные достижения

Реализованы Backend на FastAPI, Frontend на Vue 3 + TypeScript и надёжная работа с PostgreSQL. Настроены миграции, запуск и базовые сценарии тестирования.

Влияние проекта

Система упрощает работу с недвижимостью, ускоряет обработку заявок и создаёт основу для прозрачного и масштабируемого цифрового процесса.

Следующие шаги

Планируется развитие email-уведомлений, загрузки документов, расширенной аналитики и интеграции с платёжными сервисами.



Проект готов к дальнейшему развитию и может масштабироваться под новые бизнес-задачи.